

Gender-Stratified Unsupervised Discovery of Latent Socio-Economic Profiles in the Adult Income Dataset

This notebook is a clean rerun version prepared to match the final report and to avoid variable-order errors.

1. Problem Statement

The Dataset

This analysis uses the Adult Income dataset. The original source of the dataset is the UCI Machine Learning Repository, while the working CSV file used here was downloaded from a Kaggle-hosted packaged version of the Adult dataset.

- Original source: <https://archive.ics.uci.edu/dataset/2/adult>
- Download source used here: <https://www.kaggle.com/datasets/wenruiju/adult-income-dataset?resource=download>

The final workflow uses a single local CSV file named `adult_dataset.csv`.

Analysis Problem

This project investigates whether unsupervised learning can uncover meaningful latent socio-economic groups within the Adult Income dataset, and whether these structures differ across male and female subsets.

H1: Distinct socio-economic clusters exist within the data.

H2: GMM may produce more flexible and potentially more informative clustering solutions than K-Means.

H3: Gender-stratified analysis will reveal differences in cluster structure, size distribution, or profile composition across male and female subsets.

2. Import Libraries

```
In [1]: # Basic libraries
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Preprocessing and modelling
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, MinMaxScaler
from sklearn.impute import SimpleImputer
from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA

# 3D plotting
from mpl_toolkits.mplot3d import Axes3D
```

3. Set Path and Parameters

```
In [2]: DATA_PATH = 'adult_dataset.csv'
RANDOM_STATE = 42
SILHOUETTE_SAMPLE_SIZE = 5000
```

4. Load and Inspect the Dataset

```
In [3]: # Load the dataset first.
df = pd.read_csv(DATA_PATH)

print('Dataset shape:', df.shape)
df.head()
```

Dataset shape: (48842, 15)

Out[3]:

	age	workclass	fnlwgt	education	educational-num	marital-status	occupation	relationship	race	gender	capital-gain	capital-loss	hours-per-week	na col
0	25	Private	226802	11th	7	Never-married	Machine-op-inspct	Own-child	Black	Male	0	0	40	UnS
1	38	Private	89814	HS-grad	9	Married-civ-spouse	Farming-fishing	Husband	White	Male	0	0	50	UnS
2	28	Local-gov	336951	Assoc-acdm	12	Married-civ-spouse	Protective-serv	Husband	White	Male	0	0	40	UnS
3	44	Private	160323	Some-college	10	Married-civ-spouse	Machine-op-inspct	Husband	Black	Male	7688	0	40	UnS
4	18	?	103497	Some-college	10	Never-married	?	Own-child	White	Female	0	0	30	UnS

In [4]: # Define key column names early so that later cells can reuse them safely.

```
subgroup_col = 'sex' if 'sex' in df.columns else 'gender'
edu_num_col = 'educational-num' if 'educational-num' in df.columns else 'education-num'

print('Gender column used:', subgroup_col)
print('Education-number column used:', edu_num_col)
```

Gender column used: gender

Education-number column used: educational-num

In [5]: # Show a small sample for the report.

```
sample_columns = [
    'age',
    'education',
    edu_num_col,
    'marital-status',
    'occupation',
    'hours-per-week',
    subgroup_col,
    'income'
]

fig1_sample = df[sample_columns].head(8).copy()
fig1_sample
```

Out[5]:

	age	education	educational-num	marital-status	occupation	hours-per-week	gender	income
0	25	11th	7	Never-married	Machine-op-inspct	40	Male	<=50K
1	38	HS-grad	9	Married-civ-spouse	Farming-fishing	50	Male	<=50K
2	28	Assoc-acdm	12	Married-civ-spouse	Protective-serv	40	Male	>50K
3	44	Some-college	10	Married-civ-spouse	Machine-op-inspct	40	Male	>50K
4	18	Some-college	10	Never-married	?	30	Female	<=50K
5	34	10th	6	Never-married	Other-service	30	Male	<=50K
6	29	HS-grad	9	Never-married	?	40	Male	<=50K
7	63	Prof-school	15	Married-civ-spouse	Prof-specialty	32	Male	>50K

Fig. 1. Sample records from the dataset.

In [6]: # Replace '?' with missing values and inspect the dataset.

```
df = df.replace('?', np.nan)

print('Columns:')
print(df.columns.tolist())

print('\nData types:')
print(df.dtypes)

print('\nMissing values:')
display(df.isnull().sum().sort_values(ascending=False))
```

Columns:
['age', 'workclass', 'fnlwgt', 'education', 'educational-num', 'marital-status', 'occupation', 'relationship', 'race', 'gender', 'capital-gain', 'capital-loss', 'hours-per-week', 'native-country', 'income']

Data types:
age int64
workclass object
fnlwgt int64
education object
educational-num int64
marital-status object
occupation object
relationship object
race object
gender object
capital-gain int64
capital-loss int64
hours-per-week int64
native-country object
income object
dtype: object

Missing values:
occupation 2809
workclass 2799
native-country 857
fnlwgt 0
education 0
educational-num 0
age 0
marital-status 0
relationship 0
gender 0
race 0
capital-gain 0
capital-loss 0
hours-per-week 0
income 0
dtype: int64

```
In [7]: # Correlation heatmap for selected numerical variables.
numeric_cols = [c for c in ['age', 'educational-num', 'capital-gain', 'capital-loss', 'hours-per-week'] if c in df.columns]
corr = df[numeric_cols].corr()

plt.figure(figsize=(7, 5))
im = plt.imshow(corr, cmap='coolwarm', vmin=-1, vmax=1)
plt.xticks(range(len(corr.columns)), corr.columns, rotation=45, ha='right')
plt.yticks(range(len(corr.index)), corr.index)
plt.colorbar(im)
plt.title('Correlation Matrix of Selected Numerical Variables')
plt.tight_layout()
plt.show()
```

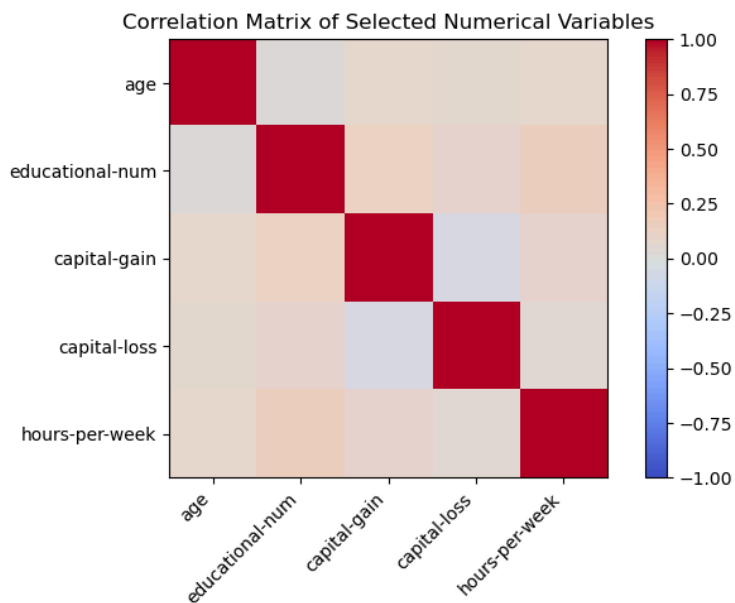


Fig. 2. Correlation matrix of selected numerical variables in the analysis dataset.

5. Pre-processing the Data

In [8]: *# Select the socio-economic variables used for clustering.*

```
feature_candidates = [
    'age',
    'workclass',
    'edu_num_col',
    'marital-status',
    'occupation',
    'relationship',
    'race' if 'race' in df.columns else None,
    'capital-gain',
    'capital-loss',
    'hours-per-week',
    'native-country' if 'native-country' in df.columns else None,
]

feature_candidates = [c for c in feature_candidates if c is not None and c in df.columns]

# Exclude target and subgroup variables from the clustering input.
exclude_cols = {'income', 'sex', 'gender'}
feature_cols = [c for c in feature_candidates if c not in exclude_cols]

print('Feature columns used for clustering:')
print(feature_cols)
```

Feature columns used for clustering:
['age', 'workclass', 'educational-num', 'marital-status', 'occupation', 'relationship', 'race', 'capital-gain', 'capital-loss', 'hours-per-week', 'native-country']

In [9]: *# Split into male and female subsets before clustering.*

```
df_male = df[df[subgroup_col] == 'Male'].copy()
df_female = df[df[subgroup_col] == 'Female'].copy()

print('Male subset shape:', df_male.shape)
print('Female subset shape:', df_female.shape)
```

Male subset shape: (32650, 15)
Female subset shape: (16192, 15)

In [10]: *# Build the preprocessing pipeline.*

```
# Numerical variables -> median imputation + MinMax scaling
# Categorical variables -> most frequent imputation + one-hot encoding

def make_preprocessor(dataframe, features):
    numeric_features = dataframe[features].select_dtypes(include=['int64', 'float64']).columns.tolist()
    categorical_features = [c for c in features if c not in numeric_features]

    numeric_transformer = Pipeline(steps=[
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', MinMaxScaler())
    ])

    categorical_transformer = Pipeline(steps=[
        ('imputer', SimpleImputer(strategy='most_frequent')),
        ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
    ])

    preprocessor = ColumnTransformer(
        transformers=[
            ('num', numeric_transformer, numeric_features),
            ('cat', categorical_transformer, categorical_features)
        ]
    )
    return preprocessor
```

In [11]: *# Check the numerical scaling step separately, following the lab style.*

```
numeric_features = df[feature_cols].select_dtypes(include=['int64', 'float64']).columns.tolist()
X_num = df[numeric_features].copy()

for col in numeric_features:
    X_num[col] = X_num[col].fillna(X_num[col].median())

scaler = MinMaxScaler().fit(X_num)
X_num_scaled = scaler.transform(X_num)
X_num_scaled = pd.DataFrame(X_num_scaled, columns=numeric_features)

X_num_scaled.describe()
```

```
Out[11]:
```

	age	educational-num	capital-gain	capital-loss	hours-per-week
count	48842.000000	48842.000000	48842.000000	48842.000000	48842.000000
mean	0.296487	0.605206	0.010791	0.020088	0.402269
std	0.187815	0.171398	0.074521	0.092517	0.126443
min	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.150685	0.533333	0.000000	0.000000	0.397959
50%	0.273973	0.600000	0.000000	0.000000	0.397959
75%	0.424658	0.733333	0.000000	0.000000	0.448980
max	1.000000	1.000000	1.000000	1.000000	1.000000

6. Learning the Clusters

```
In [12]: # Helper functions for clustering and silhouette checking.

def safe_silhouette_score(X, labels, sample_size=SILHOUETTE_SAMPLE_SIZE, random_state=RANDOM_STATE):
    if len(set(labels)) < 2:
        return np.nan
    if X.shape[0] > sample_size:
        return silhouette_score(X, labels, sample_size=sample_size, random_state=random_state)
    return silhouette_score(X, labels)

def kmeans(X, n_clusters):
    model = KMeans(
        n_clusters=n_clusters,
        init='k-means++',
        max_iter=200,
        n_init=10,
        random_state=RANDOM_STATE
    )
    model.fit(X)
    return (model.inertia_, model.cluster_centers_, model.labels_)

def gmm(X, n_clusters):
    model = GaussianMixture(
        n_components=n_clusters,
        random_state=RANDOM_STATE,
        n_init=10
    )
    model.fit(X)
    score = model.bic(X)
    labels = model.predict(X)
    return (score, model.means_, labels)
```

```
In [13]: # Transform the male and female subsets.
male_preprocessor = make_preprocessor(df_male, feature_cols)
X_male = male_preprocessor.fit_transform(df_male[feature_cols])

female_preprocessor = make_preprocessor(df_female, feature_cols)
X_female = female_preprocessor.fit_transform(df_female[feature_cols])

print('Male transformed shape:', X_male.shape)
print('Female transformed shape:', X_female.shape)
```

Male transformed shape: (32650, 85)
 Female transformed shape: (16192, 85)

6.1 Choosing the Number of Clusters

```
In [14]: # Run K-Means and GMM across candidate values of k for the male subset.
k_values = range(2, 9)

male_kbest_scores = []
male_gmm_scores = []
male_kmeans_models = {}
male_gmm_models = {}

for i in k_values:
    score, centres, labels = kmeans(X_male, i)
    sizes = pd.Series(labels).value_counts().to_dict()
    print('KMeans has {} clusters with sizes {} with score {:.2f}'.format(i, sizes, score))
    male_kbest_scores.append(score)
    male_kmeans_models[i] = (score, centres, labels)

    score, centres, labels = gmm(X_male, i)
    sizes = pd.Series(labels).value_counts().to_dict()
    print('GMM has {} clusters with sizes {} with score {:.2f}\n'.format(i, sizes, score))
```

```
male_gmm_scores.append(score)
male_gmm_models[i] = (score, centres, labels)
```

KMeans has 2 clusters with sizes {0: 19892, 1: 12758} with score 72822.83
 GMM has 2 clusters with sizes {0: 19894, 1: 12756} with score -10480154.72

KMeans has 3 clusters with sizes {0: 13590, 2: 12747, 1: 6313} with score 67251.47
 GMM has 3 clusters with sizes {1: 19074, 2: 12169, 0: 1407} with score -12878641.25

KMeans has 4 clusters with sizes {2: 13549, 1: 7582, 0: 6320, 3: 5199} with score 61918.76
 GMM has 4 clusters with sizes {1: 17995, 3: 10740, 2: 2015, 0: 1900} with score -13530754.82

KMeans has 5 clusters with sizes {3: 13564, 2: 6308, 1: 4936, 4: 4089, 0: 3753} with score 58707.62
 GMM has 5 clusters with sizes {0: 12924, 1: 6827, 2: 6013, 3: 5584, 4: 1302} with score -14981710.99

KMeans has 6 clusters with sizes {3: 11621, 1: 5005, 0: 4934, 5: 4089, 2: 3753, 4: 3248} with score 56472.15
 GMM has 6 clusters with sizes {2: 11124, 4: 6787, 0: 6259, 1: 4205, 5: 2344, 3: 1931} with score -15605889.86

KMeans has 7 clusters with sizes {3: 8074, 2: 5111, 1: 4488, 5: 4463, 6: 3907, 4: 3748, 0: 2859} with score 53852.03
 GMM has 7 clusters with sizes {3: 11139, 2: 5950, 6: 4089, 0: 3880, 4: 2971, 1: 2731, 5: 1890} with score -17438656.64

KMeans has 8 clusters with sizes {1: 6601, 2: 4921, 3: 4089, 7: 3958, 4: 3755, 0: 3725, 5: 2859, 6: 2742} with score 52155.25
 GMM has 8 clusters with sizes {2: 12197, 6: 4089, 7: 3843, 3: 3733, 0: 3619, 5: 2243, 1: 1681, 4: 1245} with score -18410800.06

```
In [15]: # Run K-Means and GMM across candidate values of k for the female subset.
female_kbest_scores = []
female_gmm_scores = []
female_kmeans_models = {}
female_gmm_models = {}

for i in k_values:
    score, centres, labels = kmeans(X_female, i)
    sizes = pd.Series(labels).value_counts().to_dict()
    print('KMeans has {} clusters with sizes {} with score {:.2f}'.format(i, sizes, score))
    female_kbest_scores.append(score)
    female_kmeans_models[i] = (score, centres, labels)

    score, centres, labels = gmm(X_female, i)
    sizes = pd.Series(labels).value_counts().to_dict()
    print('GMM has {} clusters with sizes {} with score {:.2f}\n'.format(i, sizes, score))
    female_gmm_scores.append(score)
    female_gmm_models[i] = (score, centres, labels)
```

KMeans has 2 clusters with sizes {1: 8945, 0: 7247} with score 45449.17
 GMM has 2 clusters with sizes {1: 9398, 0: 6794} with score -5569507.96

KMeans has 3 clusters with sizes {1: 7317, 2: 6372, 0: 2503} with score 40512.86
 GMM has 3 clusters with sizes {0: 7137, 1: 5870, 2: 3185} with score -6636519.25

KMeans has 4 clusters with sizes {2: 5872, 0: 4177, 1: 3705, 3: 2438} with score 36633.14
 GMM has 4 clusters with sizes {1: 6828, 3: 3898, 2: 2962, 0: 2504} with score -7925273.83

KMeans has 5 clusters with sizes {4: 4042, 3: 3668, 2: 3052, 1: 2992, 0: 2438} with score 34336.59
 GMM has 5 clusters with sizes {1: 5422, 3: 5064, 4: 2593, 2: 1661, 0: 1452} with score -7720633.00

KMeans has 6 clusters with sizes {0: 3657, 2: 3011, 4: 2959, 1: 2431, 5: 2192, 3: 1942} with score 32723.26
 GMM has 6 clusters with sizes {1: 3939, 5: 3831, 0: 3343, 4: 2559, 3: 1724, 2: 796} with score -8054189.87

KMeans has 7 clusters with sizes {1: 3160, 6: 2888, 3: 2743, 5: 2727, 4: 2428, 0: 1207, 2: 1039} with score 31681.78
 GMM has 7 clusters with sizes {6: 3118, 4: 2749, 1: 2413, 0: 2188, 2: 2141, 5: 1943, 3: 1640} with score -8123506.38

KMeans has 8 clusters with sizes {2: 3375, 5: 3051, 3: 2432, 1: 1854, 0: 1799, 7: 1410, 4: 1229, 6: 1042} with score 30614.09
 GMM has 8 clusters with sizes {0: 3166, 5: 2626, 2: 2427, 1: 2351, 4: 1578, 6: 1554, 7: 1516, 3: 974} with score -8630432.11

```
In [16]: # Plot the model-selection curves and mark the selected values.
def plot_model_selection(k_scores, g_scores, chosen_k_kmeans, chosen_k_gmm, title_prefix):
    figure, axis = plt.subplots(1, 2, figsize=(14, 5))
    k_range = list(range(2, 2 + len(k_scores)))

    axis[0].plot(k_range, k_scores)
    axis[0].scatter(chosen_k_kmeans, k_scores[chosen_k_kmeans - 2], s=200, c='red', marker='*')
    axis[0].set_title(f'{title_prefix} - The Elbow Method (KMeans)')
    axis[0].set_xlabel('Number of clusters')
    axis[0].set_ylabel('Clustering Score (Inertia)')

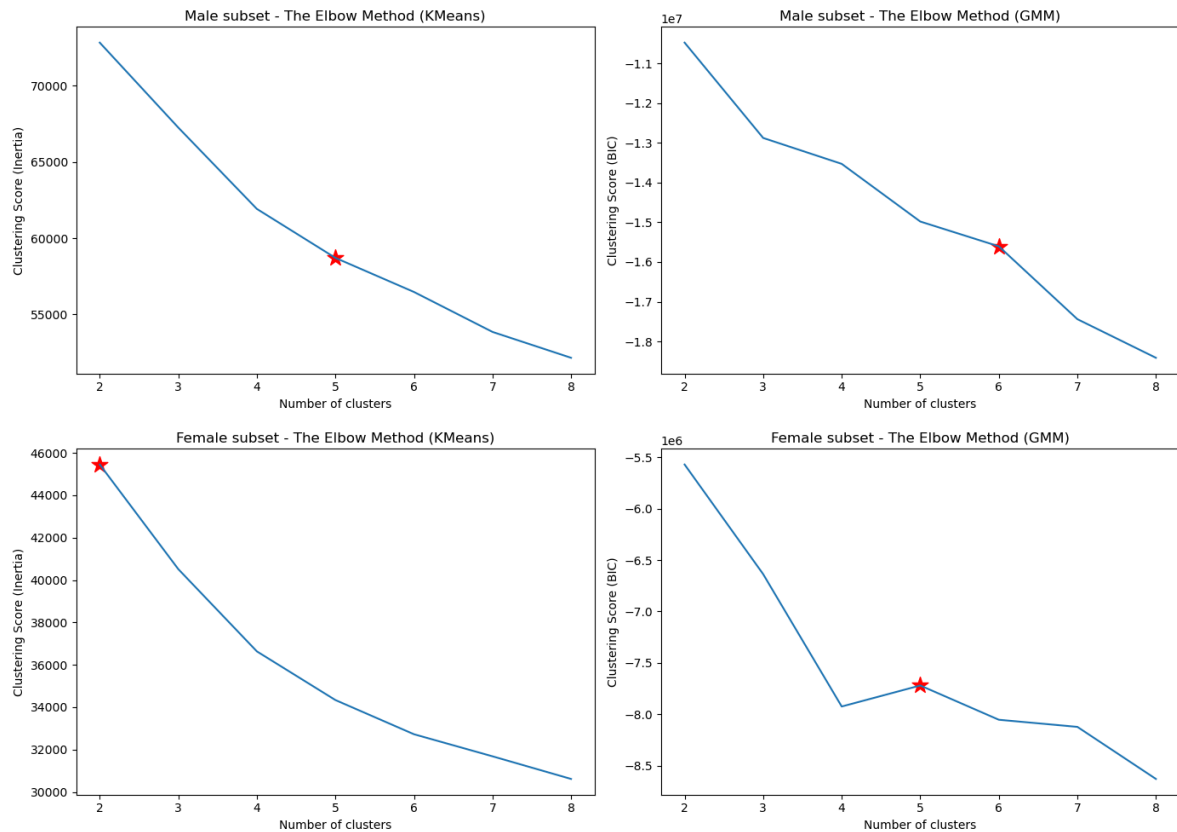
    axis[1].plot(k_range, g_scores)
    axis[1].scatter(chosen_k_gmm, g_scores[chosen_k_gmm - 2], s=200, c='red', marker='*')
    axis[1].set_title(f'{title_prefix} - The Elbow Method (GMM)')
    axis[1].set_xlabel('Number of clusters')
    axis[1].set_ylabel('Clustering Score (BIC)')

    plt.tight_layout()
```

```
plt.show()

# Selected values based on the current report logic.
male_kmeans_k = 5
male_gmm_k = 6
female_kmeans_k = 2
female_gmm_k = 5

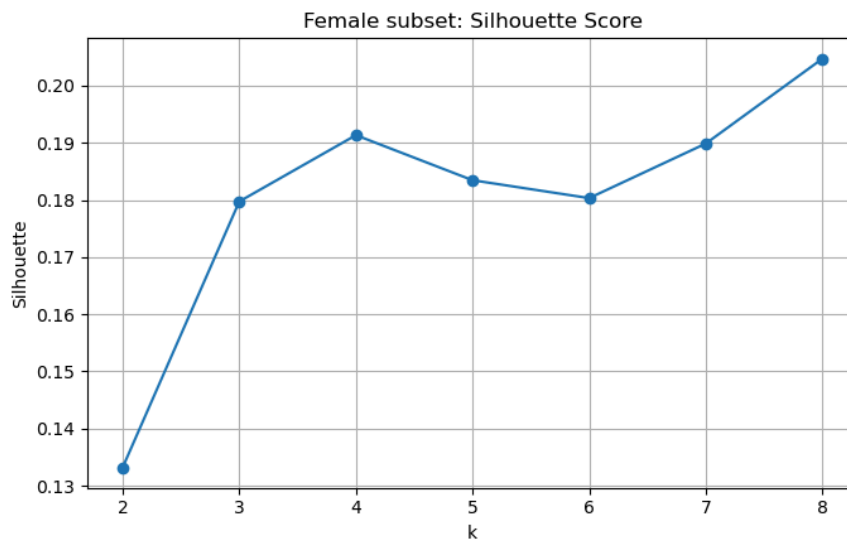
plot_model_selection(male_kbest_scores, male_gmm_scores, male_kmeans_k, male_gmm_k, 'Male subset')
plot_model_selection(female_kbest_scores, female_gmm_scores, female_kmeans_k, female_gmm_k, 'Female subset')
```



```
In [17]: # Female silhouette score is used only as an additional check.
female_silhouette_scores = []

for i in k_values:
    _, _, labels = kmeans(X_female, i)
    score = safe_silhouette_score(X_female, labels)
    female_silhouette_scores.append(score)

plt.figure(figsize=(7, 4.5))
plt.plot(list(k_values), female_silhouette_scores, marker='o')
plt.title('Female subset: Silhouette Score')
plt.xlabel('k')
plt.ylabel('Silhouette')
plt.grid(True)
plt.tight_layout()
plt.show()
```



6.2 Analyse One Set of Clusters in More Detail

```
In [18]: # Extract labels from the selected solutions.
male_kmeans_labels = male_kmeans_models[male_kmeans_k][2]
female_kmeans_labels = female_kmeans_models[female_kmeans_k][2]
male_gmm_labels = male_gmm_models[male_gmm_k][2]
female_gmm_labels = female_gmm_models[female_gmm_k][2]

In [19]: # Build profile tables for interpretation.
def profile_clusters(original_df, labels, subgroup_name):
    temp = original_df.copy()
    temp['cluster'] = labels

    numeric_summary_cols = [c for c in ['age', 'hours-per-week', 'capital-gain'] if c in temp.columns]
    if 'educational-num' in temp.columns:
        numeric_summary_cols.append('educational-num')
    elif 'education-num' in temp.columns:
        numeric_summary_cols.append('education-num')

    summary = temp.groupby('cluster')[numeric_summary_cols].mean().round(2)

    income_dist = pd.crosstab(temp['cluster'], temp['income'], normalize='index').round(3) * 100
    income_dist['Cluster Size'] = temp['cluster'].value_counts().sort_index()

    final_table = pd.concat([summary, income_dist], axis=1)

    print(f'\n{subgroup_name} subset profile:')
    display(final_table)
    return final_table

print('--- Profiling selected K-Means models ---')
male_profile_k = profile_clusters(df_male, male_kmeans_labels, 'Male (K-Means)')
female_profile_k = profile_clusters(df_female, female_kmeans_labels, 'Female (K-Means)')

print('--- Profiling selected GMM models ---')
male_profile_g = profile_clusters(df_male, male_gmm_labels, 'Male (GMM)')
female_profile_g = profile_clusters(df_female, female_gmm_labels, 'Female (GMM)')
```

--- Profiling selected K-Means models ---

Male (K-Means) subset profile:

	age	hours-per-week	capital-gain	educational-num	<=50K	>50K	Cluster Size
cluster							
0	43.74	42.66	1158.37	9.78	84.9	15.1	3753
1	25.02	35.58	174.34	9.24	98.5	1.5	4936
2	46.21	45.60	2359.87	10.84	51.1	48.9	6308
3	42.72	43.48	1489.70	10.04	57.4	42.6	13564
4	32.03	42.01	733.79	10.44	90.1	9.9	4089

Female (K-Means) subset profile:

	age	hours-per-week	capital-gain	educational-num	<=50K	>50K	Cluster Size
cluster							
0	28.08	34.94	323.53	10.18	96.4	3.6	7247
1	44.10	37.58	789.10	9.93	83.1	16.9	8945

--- Profiling selected GMM models ---

Male (GMM) subset profile:

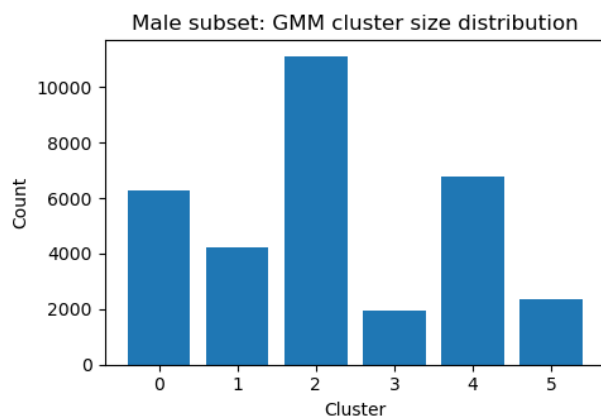
	age	hours-per-week	capital-gain	educational-num	<=50K	>50K	Cluster Size
cluster							
0	44.76	45.55	3623.93	11.38	38.2	61.8	6259
1	24.94	35.17	174.97	9.35	98.3	1.7	4205
2	42.63	42.85	692.87	9.72	62.6	37.4	11124
3	33.52	40.34	379.90	8.42	96.3	3.7	1931
4	37.39	42.41	1008.60	10.42	86.3	13.7	6787
5	47.66	46.74	1960.91	10.18	65.0	35.0	2344

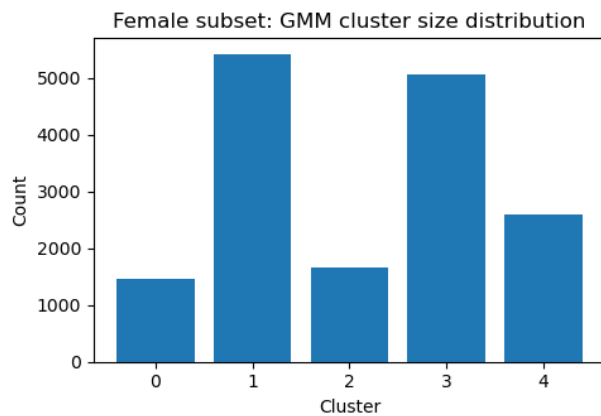
Female (GMM) subset profile:

	age	hours-per-week	capital-gain	educational-num	<=50K	>50K	Cluster Size
cluster							
0	30.11	36.33	325.17	9.93	97.9	2.1	1452
1	31.10	34.23	126.87	9.69	97.4	2.6	5422
2	43.37	37.73	646.96	9.56	88.6	11.4	1661
3	40.25	38.41	579.70	10.50	92.0	8.0	5064
4	42.31	36.20	1632.42	10.26	61.3	38.7	2593

```
In [20]: # Plot cluster-size distributions for the selected GMM solutions.
def plot_cluster_sizes(labels, title):
    counts = pd.Series(labels).value_counts().sort_index()
    plt.figure(figsize=(5, 3.5))
    plt.bar(counts.index.astype(str), counts.values)
    plt.title(title)
    plt.xlabel('Cluster')
    plt.ylabel('Count')
    plt.tight_layout()
    plt.show()

plot_cluster_sizes(male_gmm_labels, 'Male subset: GMM cluster size distribution')
plot_cluster_sizes(female_gmm_labels, 'Female subset: GMM cluster size distribution')
```





6.3 3-D Visualisation of the Clusters

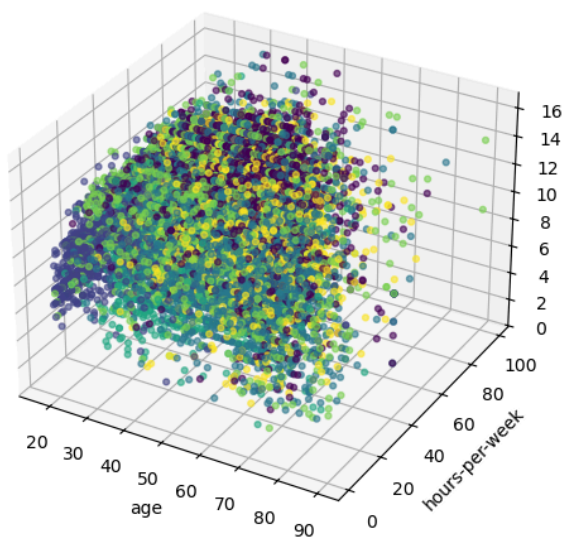
```
In [21]: # Create 3D views for the selected GMM solutions.
def plot_3d_cluster_view(original_df, labels, subgroup_name):
    plot_cols = [c for c in ['age', 'hours-per-week', 'edu_num_col'] if c in original_df.columns]
    if len(plot_cols) < 3:
        print('Not enough columns for 3D plotting.')
        return

    temp = original_df[plot_cols].copy()
    temp['cluster'] = labels

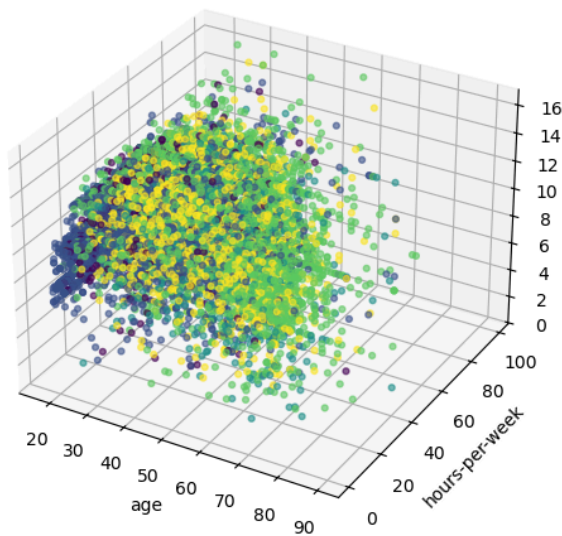
    fig = plt.figure(figsize=(7, 5))
    ax = fig.add_subplot(111, projection='3d')
    ax.scatter(
        temp[plot_cols[0]],
        temp[plot_cols[1]],
        temp[plot_cols[2]],
        c=temp['cluster'],
        s=12,
        alpha=0.6
    )
    ax.set_title(f'{subgroup_name}: 3D Cluster View')
    ax.set_xlabel(plot_cols[0])
    ax.set_ylabel(plot_cols[1])
    ax.set_zlabel(plot_cols[2])
    plt.tight_layout()
    plt.show()

plot_3d_cluster_view(df_male, male_gmm_labels, 'Male subset (GMM)')
plot_3d_cluster_view(df_female, female_gmm_labels, 'Female subset (GMM)')
```

Male subset (GMM): 3D Cluster View



Female subset (GMM): 3D Cluster View



6.4 PCA Visualisation of the Clusters

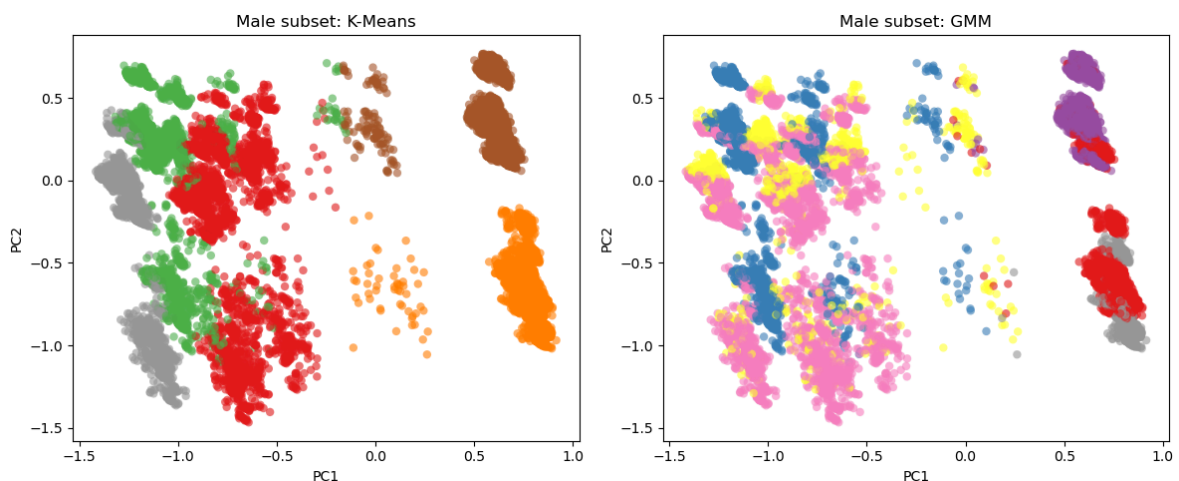
```
In [22]: # Use PCA only for visualisation.
def plot_pca_comparison(X, labels_left, labels_right, subgroup_name, left_title='K-Means', right_title='GMM'):
    X_pca = PCA(n_components=2).fit_transform(X)
    kwargs = dict(cmap=plt.colormaps['Set1'], edgecolor='none', alpha=0.6)

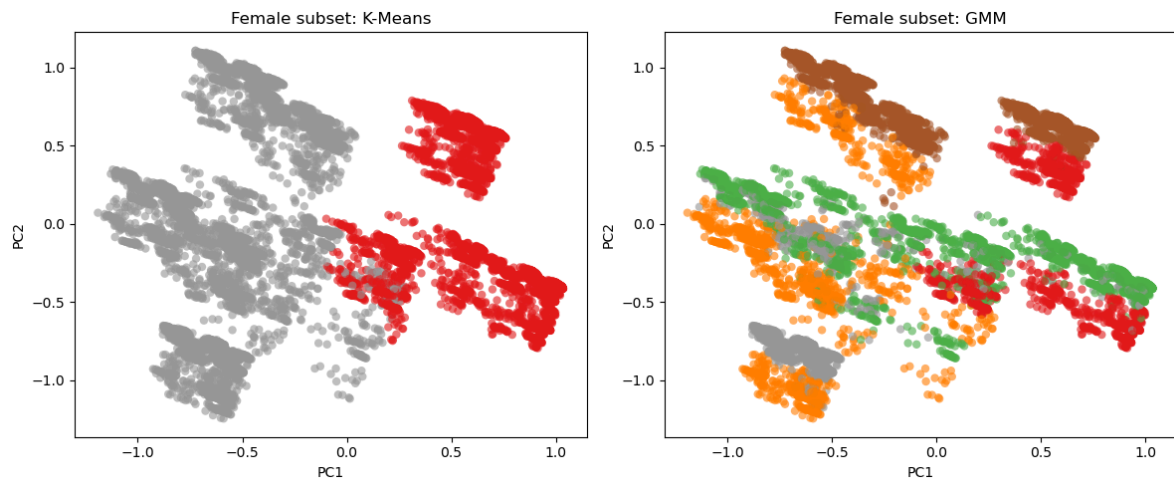
    fig, ax = plt.subplots(1, 2, figsize=(12, 5))
    ax[0].scatter(X_pca[:, 0], X_pca[:, 1], c=labels_left, **kwargs)
    ax[0].set_title(f'{subgroup_name}: {left_title}')
    ax[0].set_xlabel('PC1')
    ax[0].set_ylabel('PC2')

    ax[1].scatter(X_pca[:, 0], X_pca[:, 1], c=labels_right, **kwargs)
    ax[1].set_title(f'{subgroup_name}: {right_title}')
    ax[1].set_xlabel('PC1')
    ax[1].set_ylabel('PC2')

    plt.tight_layout()
    plt.show()

plot_pca_comparison(X_male, male_kmeans_labels, male_gmm_labels, 'Male subset')
plot_pca_comparison(X_female, female_kmeans_labels, female_gmm_labels, 'Female subset')
```





7. Outputs Used in the Report

- Fig. 1: sample records
- Fig. 2: correlation heatmap
- Male model-selection figure
- Female model-selection figure
- Female silhouette figure
- Male PCA figure
- Female PCA figure
- Male/Female cluster size plots
- Male/Female 3D plots
- Male/Female profile tables

In []: